ROS Service Communication

Prof.: Dr. Oybek Eraliev

TA: Jasurbek Mamurov

Learning Objectives:

- 📡 ROS Service Architecture
- 🔄 Request / Response Pattern
- 📝 Custom Service Messages (.srv)
- 🛠️ Server & Client Implementation
- ⚙️ Practical Examples
- 🎯 Hands-On Class Activity

ROS Communication Paradigms

Understanding where Services fit in the ROS ecosystem

Topics
(Publish/Subscribe)

Continuous data streams
One-to-many, asynchronous
Ex: sensor data, odometry

★ TODAY'S FOCUS

Services
(Request/Response)

Synchronous calls
One-to-one, blocking
Ex: move arm, set param

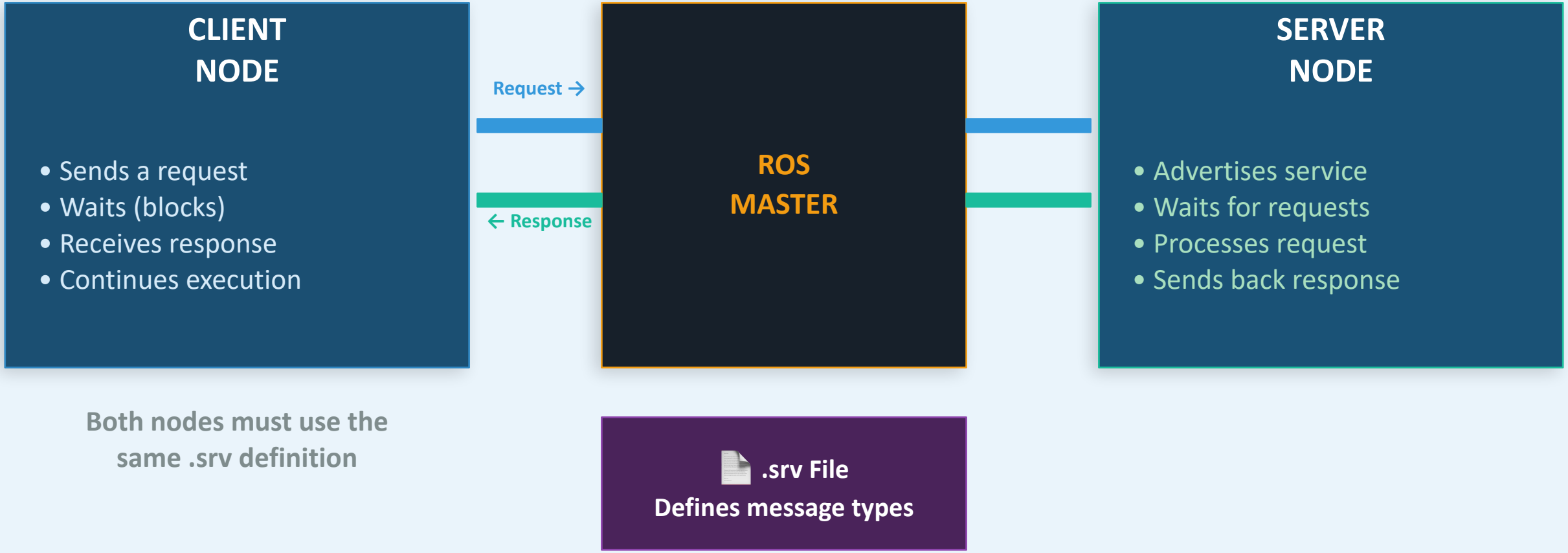
Actions
(Goal/Feedback)

Long-running tasks
Non-blocking + feedback
Ex: navigate to pose

💡 Key Insight: Use Services for operations that need a definitive response before proceeding — like asking a robot to move to a position and confirming it arrived.

ROS Service Architecture

The Request-Response communication pattern



Service Message Definition (.srv Files)

Defining the contract between client and server

AddTwoInts.srv

```
# REQUEST section
int64 a
int64 b
---
# RESPONSE section
int64 sum
```

File Location: catkin_ws/src/my_pkg/srv/
After editing CMakeLists.txt
& package.xml → catkin_make

Supported Field Types

Type	Description
`bool`	True/False
`int8, int16, int32, int64`	Integers
`uint8, uint16, uint32, uint64`	Unsigned Int
`float32, float64`	Floating Point
`string`	Text
`Header`	ROS standard header
`geometry_msgs/Pose`	Nested messages
`int32[]`	Arrays

Implementing a Service Server (Python)

Writing the node that handles requests

```python
#!/usr/bin/env python3
import rospy
from beginner_tutorials.srv import AddTwoInts, AddTwoIntsResponse

def handle_add_two_ints(req):
    result = req.a + req.b
    rospy.loginfo(f"Returning [{req.a} + {req.b} = {result}]")
    return AddTwoIntsResponse(result)

def add_two_ints_server():
    rospy.init_node('add_two_ints_server')

    # Advertise the service
    s = rospy.Service('add_two_ints', AddTwoInts,
                      handle_add_two_ints)
    rospy.loginfo("Ready to add two ints.")
    rospy.spin()  # Keep node alive

if __name__ == "__main__":
    add_two_ints_server()
```

① Import the srv type

② Callback receives Request object

③ Access fields with req.fieldname

④ rospy.Service() creates the server

⑤ spin() keeps node running

```
$ rosrun my_pkg add_two_ints_server.py
```

Implementing a Service Client (Python)

Writing the node that makes requests

```python
#!/usr/bin/env python3
import rospy
from beginner_tutorials.srv import AddTwoInts

def add_two_ints_client(x, y):
    rospy.wait_for_service('add_two_ints')
    try:
        add_two_ints = rospy.ServiceProxy(
            'add_two_ints', AddTwoInts)

        resp = add_two_ints(x, y)
        return resp.sum

    except rospy.ServiceException as e:
        rospy.logerr(f"Service call failed: {e}")

if __name__ == "__main__":
    rospy.init_node('add_two_ints_client')
    result = add_two_ints_client(3, 7)
    rospy.loginfo(f"3 + 7 = {result}")
```

① Import srv type (only Request needed)

② ALWAYS wait for service to be ready

③ Create a proxy that acts like a function

④ Call it — this BLOCKS until response arrives

⑤ Access response fields with resp.fieldname

```
$ rosrun my_pkg add_two_ints_client.py
```

Practical Example: Robot Arm Control Service

Real-world robotics application

MoveArm.srv

```
# Request: Target joint angles
float64 joint1_angle
float64 joint2_angle
float64 joint3_angle
string  mode        # "fast" or "slow"
---
# Response
bool    success
string  message
float64 time_elapsed
```

💡 Design Tip: Always include a success flag and message in responses. This allows the client to handle errors gracefully.

Client Usage

```
rospy.wait_for_service('move_arm')
move_arm = rospy.ServiceProxy(
    'move_arm', MoveArm)

resp = move_arm(
    joint1_angle=1.57,
    joint2_angle=-0.5,
    joint3_angle=0.3,
    mode="slow"
)

if resp.success:
    print(f"Done in {resp.time_elapsed}s")
else:
    print(f"Error: {resp.message}")
```

Working with Services: rosservice CLI

Debugging and testing services from the terminal

```
$ rosservice list
```
List all active services — 1

```
$ rosservice info /add_two_ints
```
Show service info (type, args, URI) — 2

```
$ rosservice call /add_two_ints 3 5
```
Call a service directly from terminal — 3

```
$ rosservice type /add_two_ints
```
Show the .srv message type used — 4

```
$ rossrv show beginner_tutorials/AddTwoInts
```
Show .srv file contents — 5

Working with Services: rosservice CLI

Debugging and testing services from the terminal

```
# Syntax: x  y  theta(radians)
rosservice call /turtle1/teleport_absolute 5.0 5.0 0.0
```

1

```
# Syntax: linear(forward)  angular(turn in radians)
rosservice call /turtle1/teleport_relative "linear: 2.0 angular: 1.5708"
```

2

```
# RGB values 0-255, width in pixels, off: 0=drawing 1=lifted
rosservice call /turtle1/set_pen "r: 255 g: 0 b: 0 width: 3 off: 0”
```

3

```
rosservice call /turtle1/set_pen "r: 255 g: 255 b: 255 width: 2 off: 1”
rosservice call /kill "name: ‘turtle2'"
```

4

```
# Returns the name of the new turtle
rosservice call /spawn "x: 2.0 y: 8.0 theta: 0.0 name: ‘turtle2’"
```

5

Common Mistakes & Best Practices

Learn from typical errors before you make them!

❌ Forgetting wait_for_service()	⏱️ Always call rospy.wait_for_service('name') before ServiceProxy
❌ No try/except around service call	🛡️ Wrap in try/except rospy.ServiceException for robust code
❌ Long blocking operations in callback	⚡ Process fast; use actions for long tasks (navigation, manipulation)
❌ Calling services in tight loop	🔁 Services have overhead; use topics for high-frequency data (>10 Hz)
❌ Not rebuilding after .srv changes	🔨 Run catkin_make every time you edit .srv, CMakeLists.txt, or package.xml

CLASS ACTIVITY

Build Your Own ROS Service — 10 Minutes

01

Create the .srv File

Create a TemperatureConverter.srv
Request: float64 celsius
Response: float64 fahrenheit, bool valid

5 min

02

Write the Server

Implement the conversion server node
Formula: F = (C × 9/5) + 32
Add validation: reject temps below −273.15°C

10 min

03

Write the Client

Build a client that converts a list of temps
[0, 100, -40, 37, -300]
Print each result and handle errors

10 min

04

Test & Extend

Test with rosservice call from terminal

5 min

Going Further: Advanced Service Concepts

Topics to explore as you become more comfortable with ROS Services

Async Service Calls

Use rospy.ServiceProxy with persistent=True for repeated calls. Consider threading for non-blocking calls with callbacks.

Custom Message Types in .srv

Nest geometry_msgs, sensor_msgs etc. inside your .srv for complex robot data structures.

Service Timeouts

wait_for_service(timeout=5.0) prevents infinite blocking. Handle TimeoutException gracefully.

Services vs Actions

For tasks >1s or needing feedback/ cancellation (navigation, pick&place), use actionlib instead.

Parameter Server Integration

Use rosparam to configure service behavior at launch time — avoids hardcoding thresholds.

Unit Testing Services

Use rostest + rosunit to write automated tests for your service nodes and message handling.

📌 Lecture Summary: ROS Services

1	ROS Services use a synchronous Request/Response pattern — the client blocks until the server replies.
2	Service types are defined in .srv files (separated by ---). Always rebuild with catkin_make after changes.
3	Server: rospy.Service('name', Type, callback) — Client: rospy.ServiceProxy('name', Type)
4	Always wait_for_service() and wrap calls in try/except to handle failures gracefully.
5	Use Topics for high-frequency data, Services for on-demand operations, Actions for long tasks.

🚀 Next Lecture: ROS Actions (actionlib) — for preemptable, long-running robot tasks


Q
&
A

Thank you!